In [ ]:
```
#self join
```

In [1]:
```
import pandas as pd
emp = pd.read_excel("D:\\Data Engineering\\Python\\Emp_Training_Data.xlsx")
```

In [3]:
```
emp.head()
```

Out[3]:

	EmployeeID	ManagerID	Title	MaritalStatus	Gender	HireDate	Dept	Job Grade
0	1	16	Gustavo Achong	M	M	2013-02-02	Sales	Admin
1	2	6	Catherine Abel	S	M	2013-08-31	Sales	Management
2	3	12	Kim Abercrombie	M	M	2014-06-16	Finance	Admin
3	4	3	Humberto Acevedo	S	M	2014-07-10	Logistics	Admin
4	5	263	Pilar Ackerman	M	M	2014-07-16	Human Resource	Admin

In [4]:
```
emp.count()
```

Out[4]:
```
EmployeeID      50
ManagerID       50
Title           50
MaritalStatus   50
Gender          50
HireDate        50
Dept            50
Job Grade       50
dtype: int64
```

In [ ]:
```
#now I want to see who's the manager for whon.display employee,managername
```

In [5]:
```
emp_mgr = pd.merge(emp,emp,left_on = "ManagerID",right_on="EmployeeID", how = "left")
```

In [7]:
```
emp_mgr .head()
```

Out[7]:

	EmployeeID_x	ManagerID_x	Title_x	MaritalStatus_x	Gender_x	HireDate_x	Dept_x	Job Grade_x	EmployeeID_y	ManagerID_y	Title_y	Mar
0	1	16	Gustavo Achong	M	M	2013-02-02	Sales	Admin	16.0	21.0	Lili Alameda	
1	2	6	Catherine Abel	S	M	2013-08-31	Sales	Management	6.0	109.0	Frances Adams	
2	3	12	Kim Abercrombie	M	M	2014-06-16	Finance	Admin	12.0	109.0	James Aguilar	
3	4	3	Humberto Acevedo	S	M	2014-07-10	Logistics	Admin	3.0	12.0	Kim Abercrombie	
4	5	263	Pilar Ackerman	M	M	2014-07-16	Human Resource	Admin	NaN	NaN	NaN	

In [11]:
```
emp_mgr[["Title_x","Title_y"]].rename(columns = {"Title_x":"EmpName","Title_y":"ManageName"}).head()
```

Out[11]:

	EmpName	ManageName
0	Gustavo Achong	Lili Alameda
1	Catherine Abel	Frances Adams
2	Kim Abercrombie	James Aguilar
3	Humberto Acevedo	Kim Abercrombie
4	Pilar Ackerman	NaN

In [13]:
```
emp.head()
```

Out[13]:

	EmployeeID	ManagerID	Title	MaritalStatus	Gender	HireDate	Dept	Job Grade
0	1	16	Gustavo Achong	M	M	2013-02-02	Sales	Admin
1	2	6	Catherine Abel	S	M	2013-08-31	Sales	Management
2	3	12	Kim Abercrombie	M	M	2014-06-16	Finance	Admin
3	4	3	Humberto Acevedo	S	M	2014-07-10	Logistics	Admin
4	5	263	Pilar Ackerman	M	M	2014-07-16	Human Resource	Admin

In [19]:
```
emp.groupby(["MaritalStatus","Gender"])[["EmployeeID"]].count()
```

Out[19]:

		EmployeeID
MaritalStatus	**Gender**	
M	**F**	7
	M	15
S	**F**	6
	M	22

In [21]:
```
def maritalsts(maritalstatuss):
    if maritalstatuss == "M":
        return  "Married"
    else:
        return  "Single"
```

In [22]:
```
maritalsts("M")
```

Out[22]:
```
'Married'
```

In [23]:
```
emp["MaritalStatus"] = emp["MaritalStatus"].apply(maritalsts) #to updating all records in that column
```

In [25]:
```
emp.head(2)
```

Out[25]:

	EmployeeID	ManagerID	Title	MaritalStatus	Gender	HireDate	Dept	Job Grade
0	1	16	Gustavo Achong	Married	M	2013-02-02	Sales	Admin
1	2	6	Catherine Abel	Single	M	2013-08-31	Sales	Management

In [27]:
```
def genderstatus(status):
    if status == "M":
        return "Male"
    else:
        return "Female"
```

In [30]:
```
genderstatus('F')
```

Out[30]:
```
'Female'
```

In [31]:
```
emp["Gender"] = emp["Gender"].apply(genderstatus) #to updating all records in that column
```

In [32]:
```
emp.head()
```

Out[32]:

	EmployeeID	ManagerID	Title	MaritalStatus	Gender	HireDate	Dept	Job Grade
0	1	16	Gustavo Achong	Married	Male	2013-02-02	Sales	Admin
1	2	6	Catherine Abel	Single	Male	2013-08-31	Sales	Management
2	3	12	Kim Abercrombie	Married	Male	2014-06-16	Finance	Admin
3	4	3	Humberto Acevedo	Single	Male	2014-07-10	Logistics	Admin
4	5	263	Pilar Ackerman	Married	Male	2014-07-16	Human Resource	Admin

In [36]:
```
emp.groupby(["MaritalStatus","Gender"])[["EmployeeID"]].count().rename(columns = {"EmployeeID": "Employee Count"})
```

Out[36]:

		Employee Count
MaritalStatus	**Gender**	
Married	**Female**	7
	Male	15
Single	**Female**	6
	Male	22

In [37]:
```
df=pd.read_excel("D:\\Data Engineering\\Python\\Bike_Data.xlsx")
```

In [38]:
```
df.head(2)
```

Out[38]:

	Region	Country	Customer	Business Segment	Category	Model	Color	SalesDate	ListPrice	UnitPrice	OrderQty
0	North America	United States	Advanced Bike Components	Components	Road Frames	LL Road Frame	Red	2020-04-01	337.22	183.94	1
1	North America	United States	Central Discount Store	Bikes	Mountain Bikes	Mountain-100	Silver	2020-04-01	3399.99	2039.99	1

In [39]:
```
df.shape
```

Out[39]: (60919, 11)

In [40]:
```
df["Cost"] = df["UnitPrice"] * df["OrderQty"] #to add cost calculated column
```

In [41]:
```
df.shape
```

Out[41]: (60919, 12)

In [42]:
```
df["Sales"] = df["ListPrice"] * df["OrderQty"] #to add sales calculated column
```

In [43]:
```
df.shape
```

Out[43]: (60919, 13)

In [44]:
```
df["Profit"] = df["Sales"] - df["Cost"] ##to add profit calculated column
```

In [46]:
```
df.shape
```

Out[46]: (60919, 14)

In [47]:
```
df.head()
```

Out[47]:

	Region	Country	Customer	Business Segment	Category	Model	Color	SalesDate	ListPrice	UnitPrice	OrderQty	Cost	Sales	Profit
0	North America	United States	Advanced Bike Components	Components	Road Frames	LL Road Frame	Red	2020-04-01	337.22	183.94	1	183.94	337.22	153.28
1	North America	United States	Central Discount Store	Bikes	Mountain Bikes	Mountain-100	Silver	2020-04-01	3399.99	2039.99	1	2039.99	3399.99	1360.00
2	North America	United States	Leading Sales & Repair	Clothing	Jerseys	Long-Sleeve Logo Jersey	Multi	2020-04-01	49.99	28.84	6	173.04	299.94	126.90
3	North America	United States	Paint Supply	Components	Mountain Frames	HL Mountain Frame	Black	2020-04-01	1349.60	714.70	2	1429.40	2699.20	1269.80
4	North America	United States	Scooters and Bikes Store	Bikes	Road Bikes	Road-450	Red	2020-04-01	1457.99	874.79	2	1749.58	2915.98	1166.40

In [ ]:
```
#Assignment [Sales_group]
```

In [ ]:
```
0 to 5000 -- Bronze
5000 to 10000 -- Silver
10000 to 15000 -- Gold
above > 15000  -- Platinum
```

In [ ]:
```
#Santhosh's hint

1.Write a function to handle above logic
2.create a new column data["Sales_Group"] = data['Sales']apply(fun)
3.how many transctions are gold/silver/gold/platinum
```

In [48]:
```
def transstatus(salesamount):
    if salesamount <= 5000:
        return "Bronze"
    elif salesamount > 5000 and salesamount <= 10000:
        return "Silver"
    elif salesamount > 10000 and salesamount <= 15000:
        return "Gold"
    else:
        return "Platinum"
```

In [51]:
```
transstatus(16000)
```

Out[51]:
```
'Platinum'
```

In [52]:
```
df["Sales_Group"] = df["Sales"].apply(transstatus) #to apply transstatus function here
```

In [58]:
```
upby(["Sales_Group"])[["Sales_Group"]].count().rename(columns = {"Sales_Group": "Transaction Count"}).sort_values("Transactio
```

Out[58]:

Sales_Group	Transaction Count
Bronze	53168
Silver	5011
Gold	1683
Platinum	1057

In [ ]:

In [ ]: